# Veri Mühendisliği, Analiz ve Görselleştirme

Modül 2 proje raporu: veri ambarı, Python analizi, A/B testi, API veri hattı ve dashboard

**Hazırlayan:** Zeynep

**Tarih:** Ekim 2026

**Araçlar:** dbt Core · Google BigQuery · Google Cloud Shell · Python (pandas, matplotlib, seaborn, statsmodels, requests) · Google Colab · Looker Studio · Git

**Veri kaynakları:** theLook e-ticaret (BigQuery herkese açık veri seti) · Cookie Cats A/B testi (Kaggle) · Open-Meteo hava durumu API'si

## Özet


- **Veri ambarı (dbt):** Ham e-ticaret verisi source → staging → mart katmanlarına ayrıldı. 4 model ve 13 veri kalitesi testi tek komutla kuruluyor: **17/17 adım başarılı**.
- **Python analizi:** Mart tablosu Colab'a çekildi. Kategoriler satış hacmi ve kalem başına fiyata göre dört gruba ayrıldı ve her grup için öneri üretildi.
- **A/B testi:** 90.189 oyunculu bir deneyde, oyundaki kapının 40. seviyeye taşınmasının 7. gün retention'ı anlamlı şekilde düşürdüğü gösterildi (%19,02 → %18,20, p = 0,0016).
- **API veri hattı:** Open-Meteo API'sinden İstanbul'un iki yıllık günlük hava verisi çekildi, temizlendi ve BigQuery'ye yüklendi (731 satır).
- **Dashboard:** Mart tablosu Looker Studio'ya bağlandı. Toplam net gelir ve kategori kırılımı tek sayfada gösterildi.

# Kullanılan teknolojiler

| Alan | Araç | Ne için kullanıldı |
|---|---|---|
| Veri ambarı | Google BigQuery | Ham veri kaynağı, dbt modellerinin ve API verisinin yazıldığı yer |
| Dönüşüm | dbt Core 1.12 + dbt-bigquery | SQL modelleri, katmanlı mimari, otomatik testler, dokümantasyon ve lineage |
| Geliştirme ortamı | Google Cloud Shell, Git | dbt projesinin kurulması, çalıştırılması ve sürüm kontrolü |
| Analiz | Python, Google Colab | BigQuery istemcisiyle veri çekme, analiz ve modelleme |
| Veri işleme | pandas | DataFrame işlemleri, groupby, tarih dönüşümleri, yeni metrikler |
| Görselleştirme | matplotlib, seaborn | Çubuk, saçılım, boxplot, histogram ve çift eksenli zaman serisi grafikleri |
| İstatistik | statsmodels | İki oranlı z-testi, p-değeri, güven aralığı |
| Veri toplama | requests (REST API) | Open-Meteo API'sinden JSON veri çekme |
| Raporlama | Looker Studio | BigQuery'ye bağlı, paylaşılabilir dashboard |

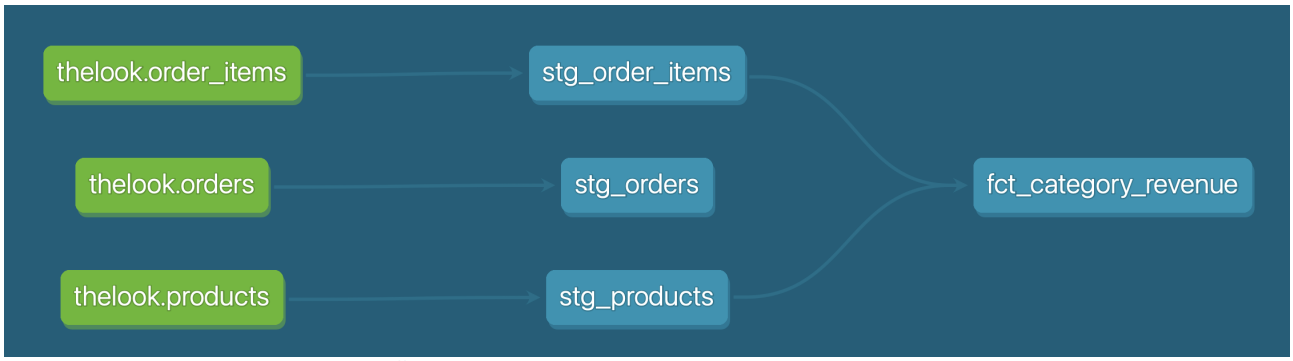
### Projelerin birbirine bağlantısı

Projeler ayrı alıştırmalar değil, tek bir veri akışının parçaları. dbt ile kurulan **fct_category_revenue** tablosu hem Python analizine hem Looker Studio dashboard'una kaynak oldu. API'den çekilen hava verisi de aynı BigQuery veri setine (dbt_zeynep) yüklendi. Böylece tüm veri tek bir ambarda, tutarlı ve test edilmiş halde duruyor.

# 1. E-ticaret veri ambarı (dbt + BigQuery)

**Problem:** Modül 1'de "iptal ve iade edilen kalemler gelire sayılmaz" kuralı her sorguda yeniden yazılıyordu ve kategori bazlı net gelir için iki sorgunun sonuçları elle birleştiriliyordu. Bu yaklaşım raporlar arasında tutarsızlığa açık.

**Çözüm:** İş kuralları dbt ile bir kez, tek yerde tanımlandı. Her model otomatik testlerle korunuyor ve modeller arasındaki bağımlılıklar belgeleniyor.



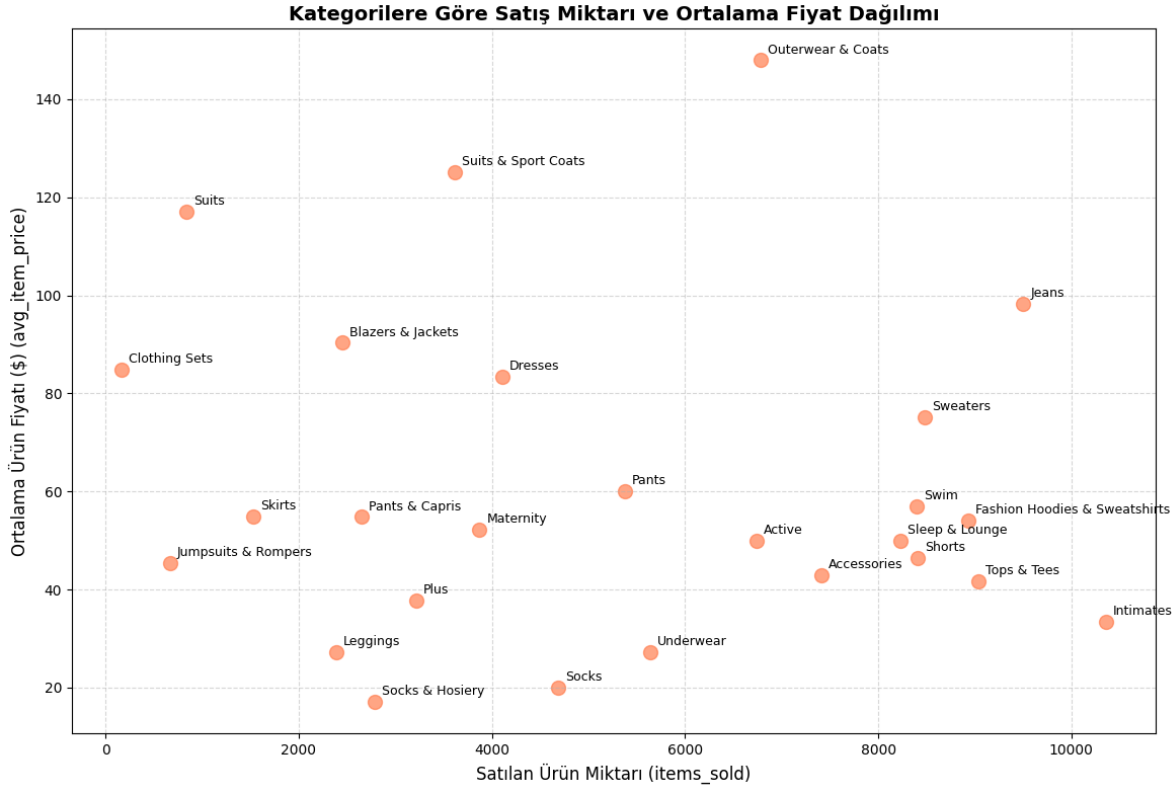

dbt docs ile üretilen lineage grafiği. Yeşil: ham kaynak tablolar, mavi: dbt modelleri.

| Katman | Model | Tür | Amaç |
|---|---|---|---|
| Staging | stg_orders | view | Siparişler: gerekli kolonlar seçildi |
| Staging | stg_order_items | view | Sipariş kalemleri: id → order_item_id |
| Staging | stg_products | view | Ürünler: id → product_id |
| Mart | fct_category_revenue | table | Kategori bazında brüt, iptal, iade ve net gelir, satılan kalem |

**Testler:** unique ve not_null testleri yalnızca tablonun gerçek anahtarına konuldu (ör. sipariş kalemlerinde order_id tekrar ettiği için unique anahtar order_item_id). accepted_values testleri sipariş durumlarını ve departman değerlerini kontrol ediyor. **dbt build sonucu: PASS=17, WARN=0, ERROR=0.**

## 2. Python ile kategori analizi (Colab)

Mart tablosu BigQuery istemcisiyle doğrudan pandas DataFrame'e çekildi. İki yeni metrik hesaplandı: **avg_item_price** (net gelir / satılan kalem) ve **loss_rate** (iptal + iade / brüt gelir). Kayıp oranı tüm kategorilerde %24-25 civarında, yani kayıp belirli kategorilere özgü değil.




26 kategori için satılan kalem sayısı ve kalem başına ortalama fiyat.

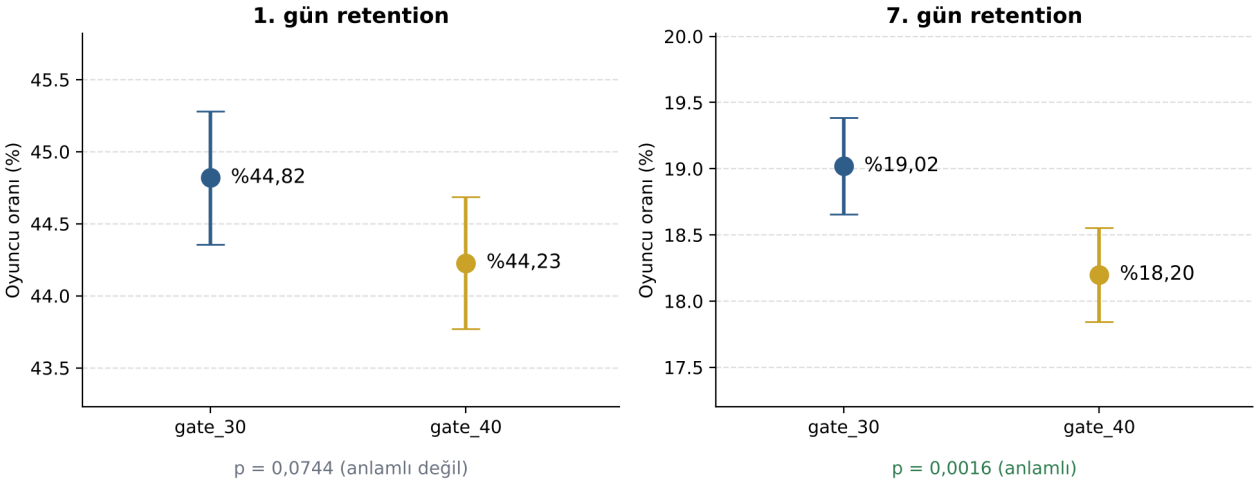
**Bulgular:** Outerwear & Coats ve Jeans hem çok satıyor hem pahalı, gelirin liderleri bunlar. Intimates en çok satan kategori (~10.362 kalem) ama ~33 $ ile net gelirde 10. sırada. Suits ve Clothing Sets az satılan ama pahalı kategoriler.

**Öneri:** Çok satan ama ucuz kategorilerde paket ve çoklu alım kampanyalarıyla sepet tutarı artırılabilir. Az satan ama pahalı kategorilerde ise hedefli reklam ve görünürlük daha uygun bir araç.

## 3. A/B testi: Cookie Cats

**İş sorusu:** Mobil oyunda oyuncuların beklemek ya da satın alma yapmak zorunda kaldığı kapı 30. seviyeden 40. seviyeye taşınırsa oyuncu elde tutma (retention) nasıl değişir? 90.189 oyuncu rastgele iki gruba ayrıldı: gate_30 (44.700) ve gate_40 (45.489).

**Yöntem:** Ana metrik 7. gün retention, yardımcı metrik 1. gün retention. Aykırı değer kontrolünde 14 günde 49.854 tur oynayan bir kullanıcı bulundu. Grupların oranları iki oranlı z-testiyle karşılaştırıldı (α = 0,05).




Noktalar retention oranını, çizgiler %95 güven aralığını gösteriyor.

| Metrik | gate_30 | gate_40 | Fark (puan) | %95 GA | p-değeri |
|---|---|---|---|---|---|
| retention_1 | %44,82 | %44,23 | 0,59 | −0,06 / 1,24 | 0,0744 |
| retention_7 | %19,02 | %18,20 | 0,82 | 0,31 / 1,33 | 0,0016 |

**Sonuç:** 1. gün farkı anlamlı değil, 7. gün farkı anlamlı. Kapıyı ileri almak, bir hafta sonra oyunda kalan oyuncu oranını göreli olarak ~%4,3 düşürüyor. Bu, her 100.000 oyuncuda ~820 kişi demek. **Öneri: kapı 30. seviyede kalmalı.**

# 4. API veri hattı: Open-Meteo → BigQuery

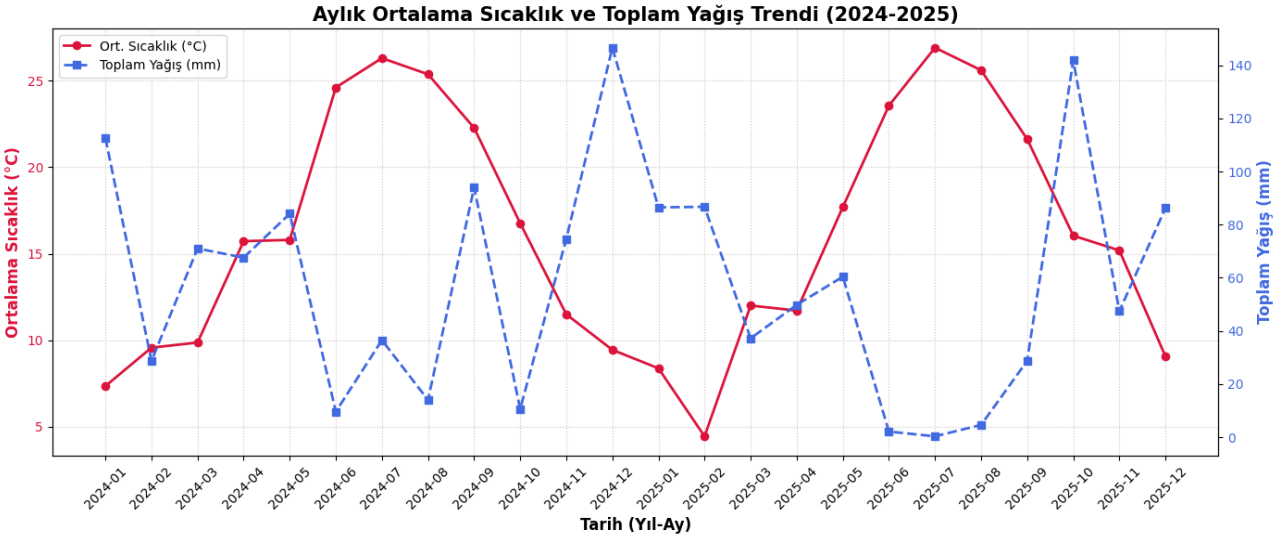
Dış bir REST API'den veri çekip temizleyen ve veri ambarına yükleyen uçtan uca bir veri hattı kuruldu. Bu veri, ilerideki makine öğrenmesi modülünde zaman serisi tahmini için kullanılacak.

```
params = {"latitude": 41.01, "longitude": 28.98,
          "start_date": "2024-01-01", "end_date": "2025-12-31",
          "daily": "temperature_2m_max,temperature_2m_min,precipitation_sum",
          "timezone": "Europe/Istanbul"}
r = requests.get("https://archive-api.open-meteo.com/v1/archive", params=params)

df = pd.DataFrame(r.json()["daily"])            # JSON -> DataFrame
df["time"] = pd.to_datetime(df["time"])
df["temp_mean"] = (df["temperature_2m_max"] + df["temperature_2m_min"]) / 2

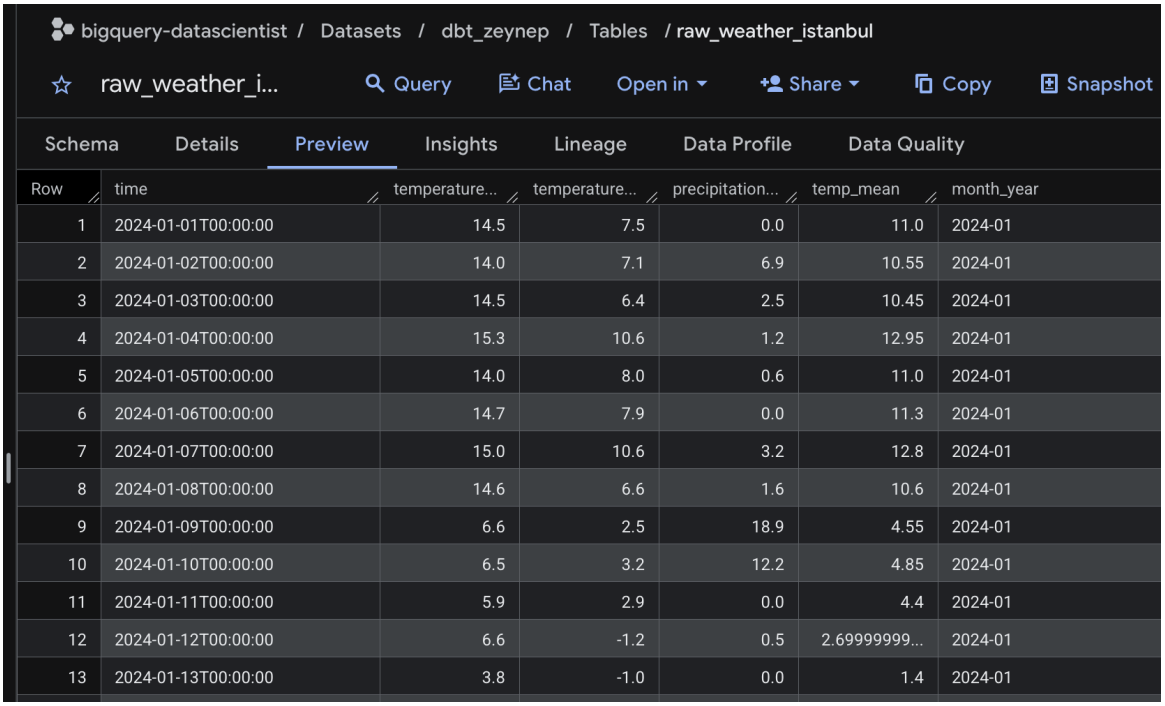
client.load_table_from_dataframe(
    df, "bigquery-datascientist.dbt_zeynep.raw_weather_istanbul")
```

- **Ayıkla:** JSON yanıtında günlük veri daily anahtarı altında, her kolon ayrı bir liste olarak geliyor. Bu yapı doğrudan DataFrame'e çevrildi.
- **Dönüştür:** Tarih kolonu datetime tipine çevrildi. Eksik değer kontrolünde 0 eksik bulundu. Günlük ortalama sıcaklık (temp_mean) eklendi.
- **Yükle:** 731 satır (2024 artık yıl: 366 + 365 gün) BigQuery'ye yazıldı.




Aylık ortalama sıcaklık ve toplam yağış (2024-2025).

**Bulgu:** 2025 yazı neredeyse kuru geçti. Haziran-ağustos toplam yağış 2025'te 7 mm, 2024'te 60 mm. Sıcaklık iki yılda da temmuzda zirve yapıyor (~26-27 °C).




| Row | time | temperature... | temperature... | precipitation... | temp_mean | month_year |
|---|---|---|---|---|---|---|
| 1 | 2024-01-01T00:00:00 | 14.5 | 7.5 | 0.0 | 11.0 | 2024-01 |
| 2 | 2024-01-02T00:00:00 | 14.0 | 7.1 | 6.9 | 10.55 | 2024-01 |
| 3 | 2024-01-03T00:00:00 | 14.5 | 6.4 | 2.5 | 10.45 | 2024-01 |
| 4 | 2024-01-04T00:00:00 | 15.3 | 10.6 | 1.2 | 12.95 | 2024-01 |
| 5 | 2024-01-05T00:00:00 | 14.0 | 8.0 | 0.6 | 11.0 | 2024-01 |
| 6 | 2024-01-06T00:00:00 | 14.7 | 7.9 | 0.0 | 11.3 | 2024-01 |
| 7 | 2024-01-07T00:00:00 | 15.0 | 10.6 | 3.2 | 12.8 | 2024-01 |
| 8 | 2024-01-08T00:00:00 | 14.6 | 6.6 | 1.6 | 10.6 | 2024-01 |
| 9 | 2024-01-09T00:00:00 | 6.6 | 2.5 | 18.9 | 4.55 | 2024-01 |
| 10 | 2024-01-10T00:00:00 | 6.5 | 3.2 | 12.2 | 4.85 | 2024-01 |
| 11 | 2024-01-11T00:00:00 | 5.9 | 2.9 | 0.0 | 4.4 | 2024-01 |
| 12 | 2024-01-12T00:00:00 | 6.6 | -1.2 | 0.5 | 2.69999999... | 2024-01 |
| 13 | 2024-01-13T00:00:00 | 3.8 | -1.0 | 0.0 | 1.4 | 2024-01 |

BigQuery'ye yüklenen raw_weather_istanbul tablosunun önizlemesi (731 satır).

**Üretim için iyileştirmeler:** Yükleme WRITE_TRUNCATE ile yapılarak tekrar çalıştırıldığında verinin ikiye katlanması önlenmeli. job.result() ile yükleme sonucu beklenmeli, r.raise_for_status() ile API hataları yakalanmalı. Bu adımlar veri hattını tekrar çalıştırılabilir (idempotent) hale getirir.

## 5. Dashboard: Looker Studio

Looker Studio doğrudan BigQuery'deki **fct_category_revenue** mart tablosuna bağlandı. Dashboard ham veriyi değil, dbt ile test edilmiş tabloyu okuyor. Böylece dashboard'daki rakamlar Python analiziyle birebir aynı: toplam net gelir **8.109.150 $**.

- Skor kartı: toplam net gelir
- Çubuk grafik: net gelire göre ilk 10 kategori
- Tablo: 26 kategorinin net geliri, büyükten küçüğe sıralı

**Canlı dashboard:** Looker Studio raporu

net_revenue
8.109.150,76

| | category | net_revenue ▾ |
|---|---|---|
| 1. | Outerwear & Coats | 1.003.876,47 |
| 2. | Jeans | 932.893,03 |
| 3. | Sweaters | 636.977,6 |
| 4. | Fashion Hoodies & Sweats… | 482.985,57 |
| 5. | Swim | 478.754,41 |
| 6. | Suits & Sport Coats | 452.559,24 |
| 7. | Sleep & Lounge | 410.897,2 |
| 8. | Shorts | 390.414,85 |
| 9. | Tops & Tees | 377.360,74 |
| 10. | Intimates | 346.395,99 |
| 11. | Dresses | 342.803,03 |
| 12. | Active | 336.856,55 |
| 13. | Pants | 322.428,74 |
| 14. | Accessories | 317.691,54 |
| 15. | Blazers & Jackets | 221.718,46 |
| 16. | Maternity | 201.862,23 |
| 17. | Underwear | 153.523,67 |
| 18. | Pants & Capris | 144.977,02 |
| 19. | Plus | 121.441,4 |
| 20. | Suits | 98.393,18 |
| 21. | Socks | 93.752,26 |
| 22. | Skirts | 84.035,55 |
| 23. | Leggings | 64.957,82 |
| 24. | Socks & Hosiery | 47.802,57 |
| 25. | Jumpsuits & Rompers | 30.125,5 |
| 26. | Clothing Sets | 13.666,14 |

Looker Studio dashboard'undan skor kartı ve kategori tablosu.

# 6. Karşılaşılan sorunlar ve öğrenilenler

| Sorun | Çözüm ve öğrenilen |
|---|---|
| dbt: unique testi tekrar eden kolonlarda başarısız olurdu | unique testi sadece tablonun gerçek anahtarına konur. Önce tablonun "bir satır neyi temsil ediyor" sorusu cevaplanmalı. |
| dbt: SQL sözdizimi hataları (eksik virgül, eksik IF argümanı) | Hata mesajındaki satır ve kelimeyi okuyup kök nedeni bulmak. İlk hata düzelince ona bağlı hatalar da kayboldu. |
| dbt 1.12 deprecation uyarısı | accepted_values testleri yeni arguments sözdizimine taşındı. |
| theLook verisinin yeniden üretilmesi | Rakamlar Modül 1'den biraz farklı. Raporlarda veri tarihini belirtmek gerekir. |
| A/B: aşırı uç değer (49.854 tur) | Uç değer ortalama ve standart sapmayı bozar. Retention (True/False) metriklerini etkilemez, ama kontrol edildiği belgelenmeli. |
| A/B: "anlamlı" demek yetmez | Etki büyüklüğü (göreli %4,3 düşüş, 100.000 oyuncuda ~820 kişi) ve güven aralığı karar vericiye asıl bilgiyi verir. |
| API: yüklemenin tekrar çalıştırılması veriyi çoğaltır | Veri hatları idempotent tasarlanmalı (WRITE_TRUNCATE). |
| Dashboard: "Diğer" grubu ve uzun kategori adları | Varsayılan grafik ayarları her zaman okunur değil. Gruplamayı kapatmak, yatay çubuk ve para birimi biçimi okunurluğu artırır. |

# 7. Kazanılan yetkinlikler

| Alan | Yetkinlik |
|---|---|
| Veri mühendisliği | Katmanlı veri modeli, dbt (source, ref, materialization), veri kalitesi testleri, lineage, REST API'den veri toplama, BigQuery'ye yükleme |
| SQL | CTE, JOIN, koşullu toplama (SUM(IF), COUNT(DISTINCT IF)), tarih fonksiyonları |
| Python | pandas ile veri temizleme ve dönüştürme, groupby ve agg, JSON işleme, BigQuery istemcisi |
| İstatistik | Hipotez testi, iki oranlı z-testi, p-değeri, güven aralığı, etki büyüklüğü, aykırı değer analizi |
| Görselleştirme | matplotlib, seaborn, Looker Studio, iş sorusuna uygun grafik seçimi |
| İş yorumu | Metrik seçimi, bulguları öneriye çevirme, karar vericiye sunum |

**Ayrıntılı raporlar:** dbt veri ambarı (9 sayfa) ve A/B testi (4 sayfa) için ayrı, ayrıntılı proje raporları da mevcuttur.